
Design Integration Solutions

Given a use case that includes technical requirements, constraints or drivers determine the standards, components, techniques, and security mechanism that should be used.

Develop your career with Salesforce Training and Certification Preparation

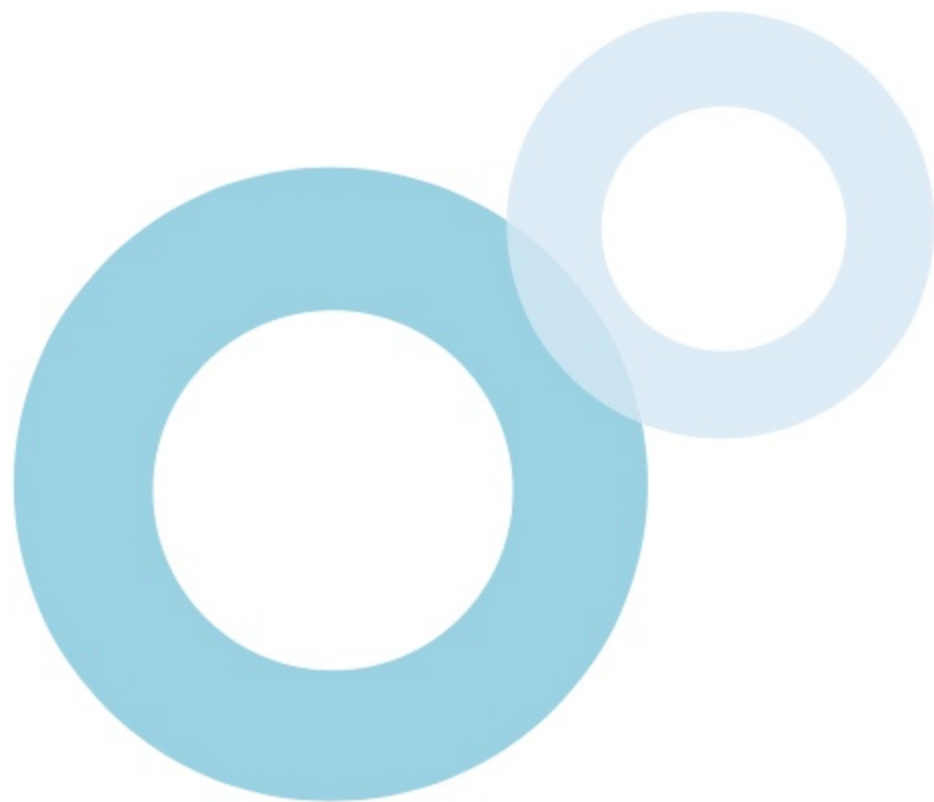
Table of Contents

-  [Commerce Cloud](#)
-  [Integration Strategies & Principles](#)
-  [Integration Security](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Identify the integration standards to which components must adhere.
-  Identify the integration techniques and calling mechanisms available for Salesforce.
-  Identify the security mechanisms applicable to integrations.
-  Identify the standards, components, techniques, and security mechanisms applicable to specific technical requirements.



Introduction

An integration solution may consist of several **components**, which can be affected by **standards, calling mechanisms**, and **security mechanisms**. Each component of an integration solution must be compatible with the standards supported by Salesforce. For example, **SOAP API calls** must use the standards supported by Salesforce (**WSDL 1.1**, **SOAP 1.1**, **WSI-Basic Profile 1.1**, and **HTTP**).

Certain **security considerations** also apply to integration solutions. For example, an **OAuth trust** must be established when using one of the **Salesforce APIs**, such as REST or SOAP API.

The **technique or calling mechanism** used for an integration solution is affected by the requirement. For example, if a remote process needs to be triggered from a Salesforce user interface, mechanisms such as an **external service** invoked from a **flow**, a **Lightning component**, or a **Visualforce page** can be used as the calling mechanism.



Overview

Integration Components

An integration solution may comprise several components, such as an HTTP request that uses a specific method for invoking a REST API on Salesforce.

Integration Techniques

The technique or calling mechanism depends on the integration solution. For example, for a simple inbound integration, REST or SOAP API can be used.



Integration Standards

The components of an integration solution should be compatible with the standards supported by Salesforce. For example, a REST API call should use one of the supported methods (GET, POST, DELETE, PATCH, and PUT).

Security Mechanisms

Certain security mechanisms apply to integration solutions. For example, a remote system must establish an OAuth trust in order to use one of the Salesforce APIs.

Integration Standards

The **components** of an integration solution should be compatible with the **standards** supported by Salesforce.

❖ SOAP CALLOUTS

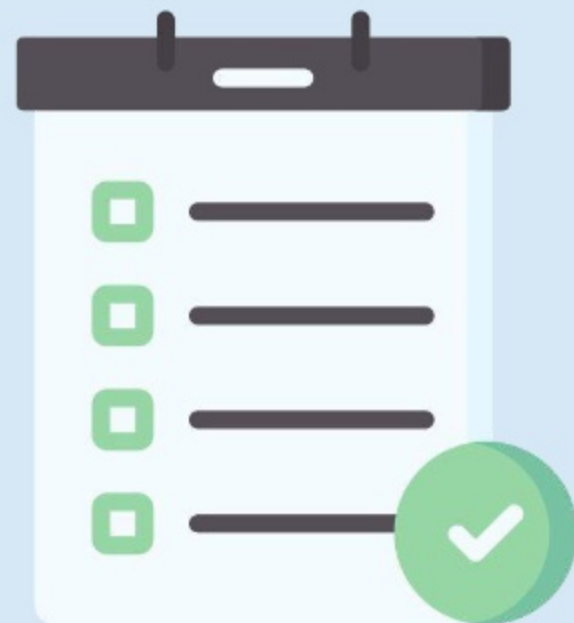
Salesforce supports these **web service standards** for SOAP callouts: **WSDL 1.1**, **SOAP 1.1**, **WSI-Basic Profile 1.1**, and **HTTP**

❖ HTTP CALLOUTS

Salesforce supports the standard HTTP **GET, PATCH, POST, PUT, and DELETE methods** to call **REST services**.

❖ OUTBOUND MESSAGING

The remote endpoint must be able to implement a **listener** that can receive messages in the predefined format (**WSDL**) sent from Salesforce. The remote listener must participate in a **contract-first implementation**, where the **contract** is supplied by Salesforce.



Integration Standards

The **components** of an integration solution should be compatible with the **standards** supported by Salesforce.

❖ PLATFORM EVENTS

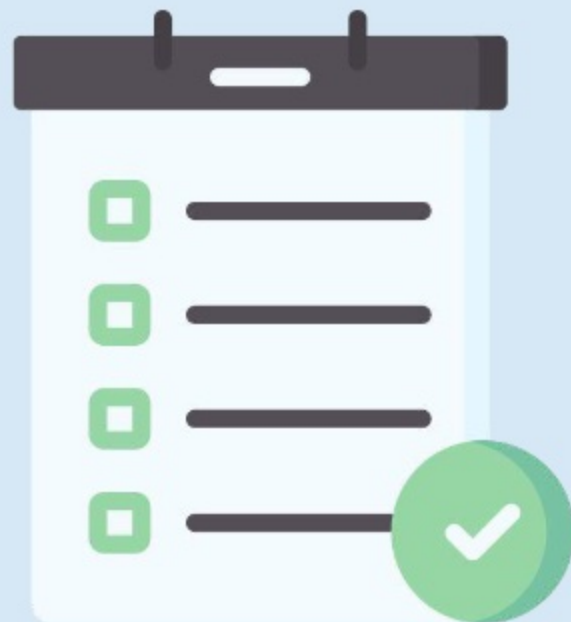
An external client must be able to use **CometD** to **subscribe** to platform events. A **custom CometD client** or an **EMP Connector** can be used.

❖ SOAP API

The remote system must be able to implement a client that can invoke the Salesforce SOAP API. The client must participate in a **contract-first implementation** and should be able to use either the **Salesforce enterprise or partner WSDL** to generate client stubs for invoking the SOAP API.

❖ REST API

The remote system must be able to implement a client that can invoke Salesforce's REST services. The client should be able to use **OAuth 2.0** or **username and password** for **authentication**.



Integration Standards

The **components** of an integration should be compatible with the **standards** supported by Salesforce.

❖ APEX WEB SERVICE

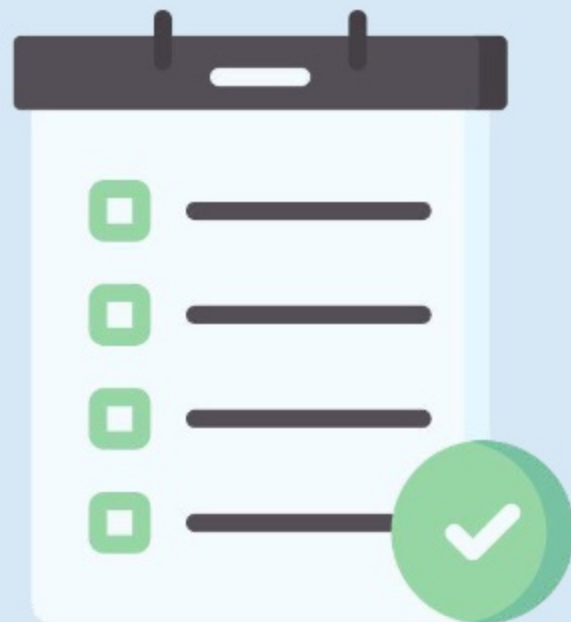
A remote system should be able to consume a **custom Apex web service WSDL** to generate client stubs for invoking the web service. The **webservice** keyword should be used to define its methods.

❖ APEX REST SERVICE

The **resource URIs** and **applicable verbs** should be defined in an **Apex REST service** using suitable annotations, including **@RestResource**, **@HttpGet**, **@HttpPost**, and others.

❖ SALESFORCE CONNECT

Data in the remote system should be exposed via **OData (Open Data) 2.0 or 4.0** producers. If the remote system is another Salesforce org, Lightning Platform REST API is used to access the data. The **Apex Connector Framework** can be used to develop a custom adapter.



Integration Techniques or Calling Mechanisms

The **technique or calling mechanism** depends on the integration solution.

❖ USER INTERFACE

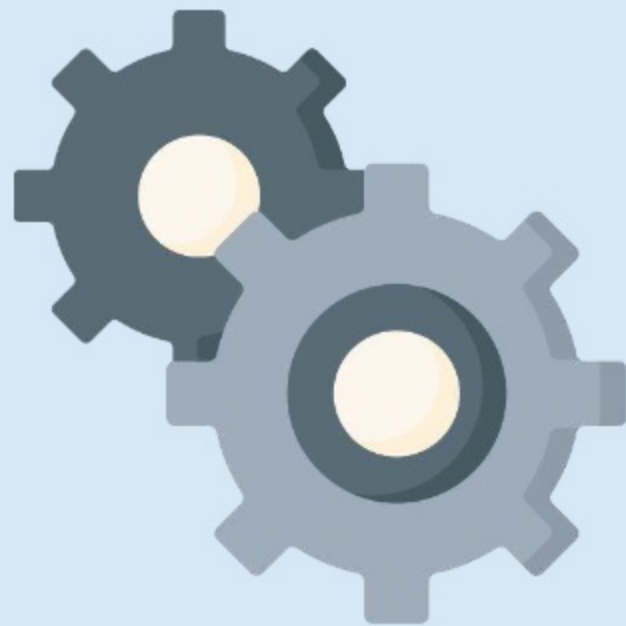
If a remote process needs to be triggered from the user interface, mechanisms such as an **enhanced external service invoked from a flow**, a **Lightning component**, or a **Visualforce page** can be used as the calling mechanism.

❖ DML EVENTS

If a remote process needs to be invoked from a DML event, a **flow** or an **Apex trigger** can be used.

❖ BATCH PROCESSING

A **batch Apex** class can be used for batch processing a **remote process** in an external system.



Integration Techniques or Calling Mechanisms

The **technique or calling mechanism** depends on the integration solution.

❖ INBOUND SIMPLE REQUESTS

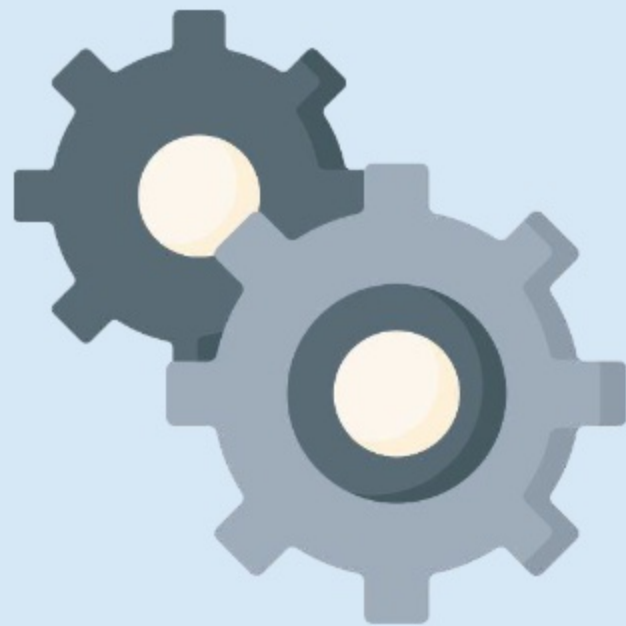
If a remote process needs to perform a **simple operation** in Salesforce, such as retrieving or creating a record, the **REST/SOAP API** can be used.

❖ INBOUND COMPLEX REQUESTS

If a remote process needs to perform a **complex operation** in Salesforce, a custom **Apex web service**, an **Apex REST service**, or a **flow invoked via REST API** can be used.

❖ LARGE DATA VOLUME REQUESTS

If a remote process needs to invoke an operation in Salesforce that involves a **large number of records**, such as inserting millions of account records, **Bulk API 2.0** should be used.



Integration Security Mechanisms

The **security mechanisms** ensure that any call to or from a remote system maintains the **confidentiality, integrity, and availability** of the request.

❖ APEX HTTP/SOAP CALLOUTS

A callout to a remote system uses **one-way SSL** by default, but **two-way SSL** is supported with both **self-signed** and **CA-signed certificates**. **WS-Security** is not supported when generating the Apex proxy class. **One-way hashes** or **digital signatures**, implemented using the **Apex Crypto** class methods, can be used to maintain the integrity of the request. The remote system should be protected by a **firewall**.

❖ OUTBOUND MESSAGING

An outbound message uses a **one-way SSL** by default, but **two-way SSL** can be used with the **outbound messaging certificate**. **Salesforce server IP ranges** should be **whitelisted** on remote integration servers. The remote system should be protected by a **firewall**.



Integration Security Mechanisms

The **security mechanisms** ensure that any call to or from a remote system maintains the **confidentiality, integrity, and availability** of the request.

❖ PLATFORM EVENTS

To **publish** an event in Salesforce, a user must have **Create** access to the event entity, and to **subscribe** to the event, the user must have **Read** access. An **external system** that needs to **subscribe** to a platform event should be able to **authenticate to the Streaming API**.

❖ REST API

A remote system must establish an **OAuth trust for authorization** before making REST calls using HTTP verbs. A **valid session ID** can also be obtained via **SOAP API** or an **outbound message**. To **maximize performance**, it is recommended that an external client calling the REST API **cache** and **reuse** the **session ID**.



Integration Security Mechanisms

The **security mechanisms** ensure that any call to or from a remote system maintains the **confidentiality, integrity, and availability** of the request.

❖ SOAP API

The remote system should be able to log in using **valid credentials** to obtain a **session ID** before making SOAP calls. The session ID can also be obtained using an **outbound message**. To **maximize performance**, it is recommended that the external client **cache** and **reuse** the **session ID**.

❖ APEX WEB/REST SERVICE

To invoke an **Apex web service** or **Apex REST service**, the remote system must establish an **OAuth trust** for authorization.

❖ BULK API 2.0

To make **Bulk API 2.0** calls, the remote system must establish an **OAuth trust** for authorization.



Commerce Cloud

[Table of Contents](#)



Commerce Cloud

Salesforce Commerce Cloud provides comprehensive **ecommerce solutions** for B2C and B2B businesses, enabling efficient **online shopping experiences** for its customers.



OVERVIEW

Commerce Cloud, a core part of the **Salesforce Customer Success Platform**, provides ecommerce solutions for **B2C** and **B2B customers**, enabling organizations to create exceptional online shopping experiences for both **consumers** and **businesses**.



ORIGINS

Commerce Cloud was established in **2016** with Salesforce's acquisition of **Demandware**, which subsequently formed the basis of **Salesforce B2C Commerce**. In **2018**, its capabilities expanded to include B2B through the acquisition of **CloudCraze**, which became Salesforce **B2B Commerce**.



CAPABILITIES

Commerce Cloud includes three core offerings: **B2B Commerce**, **B2C Commerce**, and **B2B2C Commerce**. These capabilities empower organizations to develop and implement ecommerce strategies at their own pace, tailored to unique needs.

B2B Commerce

Salesforce B2B Commerce streamlines large-scale business purchases with **tailored storefronts**, **contract pricing**, and **CRM integration** across various industries, including manufacturing and healthcare

❖ LARGE-SCALE

B2B Commerce supports major companies making **bulk online purchases** across industries like manufacturing, consumer goods, technology, and healthcare.

❖ KEY FEATURES

B2B Commerce sites handle large-scale transactions with features like **authenticated access**, **custom storefronts**, **account-specific catalogs**, **advanced shipping**, **multiple payment options**, and **quick reordering**.

❖ INTEGRATION

B2B Commerce, built on the Salesforce Lightning Platform, integrates with **Salesforce CRM** for unified management. Buyers can manage their **accounts** and **orders** directly through ecommerce portals.



B2C Commerce

B2C Commerce enables brands to launch and manage **multiple online stores** across **channels** and **regions**, while integrating marketing solutions for a personalized, seamless **shopper experience**.

❖ **MANAGE ONLINE EXPERIENCE**

B2C Commerce lets brands build and coordinate **online experiences across digital channels**. Brands can launch and manage **responsive sites** with rich **storefronts**, **shopping carts**, and **customizations**.

❖ **MULTIPLE BRANDS & GEOGRAPHIES**

Retailers often need **different sites** for **multiple brands** or **markets**. Commerce Cloud lets them launch and manage multiple sites in **one platform**.

❖ **INTEGRATING TOUCHPOINTS**

Shoppers engage through **email**, **social media**, and in **stores**. By integrating ecommerce with marketing solutions, brands can personalize **interactions** and expand store **inventory**.



B2B2C Commerce

B2B2C Commerce enables **B2B merchants** to quickly launch **direct-to-consumer channels** with intelligent experiences, unified operations, and a connected customer journey.

❖ OPTIMIZE DELIVERY

The platform enables the **launch** of streamlined commerce site, using intuitive **drag-and-drop** tools on a dependable, scalable platform.

❖ IMPROVE EXPERIENCE

Personalization is boosted by leveraging **Einstein Recommendations** and **Search**, all with simple drag-and-drop functionality.

❖ CENTRALIZED OPERATION

B2B, **B2C**, **Order Management**, and **partner ecosystems** are combined into **one shared data model** for a single source of truth.

❖ CUSTOMER JOURNEY

Digital transformation is **accelerated** with a CRM-powered ecosystem that seamlessly connects **marketing**, **sales**, **commerce**, and **service**.



Integration Strategies & Principles

Table of Contents 

Integration Strategies

Integration strategies in B2C Commerce involve choosing between **synchronous** vs. **asynchronous**, **push** vs. **pull**, and **client-side** vs. **server-side** approaches, tailoring each to the unique requirements.



SYNCHRONOUS

Synchronous integration ensures **real-time data updates** and is ideal for stateless third-party integrations like loyalty points, tax calculation, and order history. For efficiency, it is recommended to send comprehensive requests (e.g., entire baskets for tax calculations) to avoid the complexities and limitations of stateful, multi-request processes.



ASYNCHRONOUS

Asynchronous integration processes data at scheduled times, making it ideal when **real-time updates are not needed**, and is implemented via scheduled jobs, file transfers, or API calls, often during off-peak hours for optimal performance.

Integration Strategies

Integration strategies in B2C Commerce involve choosing between **synchronous** vs. **asynchronous**, **push** vs. **pull**, and **client-side** vs. **server-side** approaches, tailoring each to the unique requirements.



PUSH

Push integration is **initiated by the data source**, such as a CRM system sending customer data changes to B2C Commerce using the Data API, or Salesforce Change Data Capture pushing record updates to an external system to synchronize data.



PULL

Pull integration is **initiated by the target destination**, such as B2C Commerce requesting customer data file from an SFTP (Secure Shell File Transfer Protocol) server, or Apex invocation that performs a callout to a third-party API to retrieve weather data.

Integration Strategies

Integration strategies in B2C Commerce involve choosing between **synchronous** vs. **asynchronous**, **push** vs. **pull**, and **client-side** vs. **server-side** approaches, tailoring each to the unique requirements.



CLIENT-SIDE INTEGRATION

The client-side refers to the browser layer of a platform. Within B2C Commerce, third-party tracking is handled here, and many third-party systems offer direct client-side integration options. Examples include **ratings/reviews**, **captcha verification**, and **maps integration**. Client-side integrations provide an **asynchronous experience**, often implemented using **Ajax**.



SERVER-SIDE INTEGRATION

The server-side refers to, for example, the B2C backend custom layer. This approach is suitable when accessing a **third-party service** that requires a **username and password** or **secret token**. Server-side integration is also ideal for **persisting data** or including **third-party data** in search indexes.

Integration Principles

Applying integration principles like **loose coupling** for independence and fault tolerance, and using a **transfer server** for secure, maintainable data exchanges, ensures a scalable and robust platform.



LOOSE COUPLING

Loose coupling **minimizes dependency** between system components, thereby making them **more independent** and **less prone** to breaking when changes occur, while also improving **fault tolerance** and reducing the impact of **integration failures**. For example, publishers and subscribers are decoupled, meaning the publisher doesn't need to know which systems or processes will consume the platform events.



TRANSFER SERVER

In B2C Commerce, a transfer server facilitates **secure data exchanges** between the platform and third parties, supporting various data types (e.g., product, customer, coupon) with decoupled processes for **maintainability**. Common options include **SFTP**, **HTTP file transfer methods**, and others, along with considerations for **PCI/PII** compliance and data encryption.

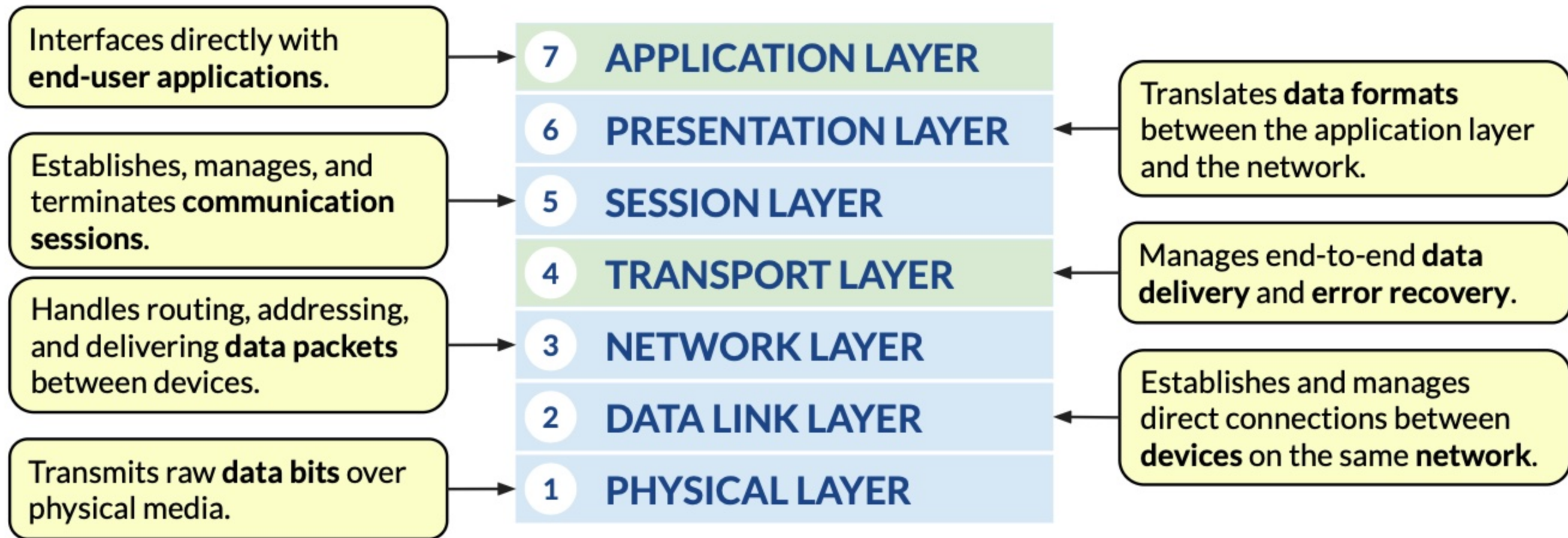
Integration Security

[Table of Contents](#)



Open Systems Interconnection

The OSI model is a **seven-layer architecture** used to **modularize** network functions, which enhances **security** and enables **targeted encryption**, **secure session management**, and **reliable data delivery**.



Integration Security

In B2C Commerce, **third-party integration** primarily targets the **transport layer** and **application layer** of the Open Systems Interconnection model.



TRANSPORT LAYER

- A dedicated **firewall link** is established between the third party and B2C Commerce.
- Transport layer **certificates** are utilized to authenticate **both sides** in a transaction.
- Data transmission is secured by employing **HTTPS encryption**.
- The client's **private key** is **automatically applied** based on its associated hostname when an HTTPS web service is called.



APPLICATION LAYER

- **Messages** to web services and **data in files** sent to third parties should be **encrypted**.
- Certificates are stored in a **cartridge** (code & data container) are used to **automatically encrypt** and **decrypt SOAP messages** for web services.
- Authentication request headers are used with **REST APIs**. **Encoded username-password** is used in basic authentication, and **bearer tokens that expire** are used in OAuth.

Security Protocols

Secure protocols like **HTTPS**, **SFTP**, **Basic Auth**, **OAuth**, **TLS**, and **SOAP signing** should be used for third-party app integration with B2C Commerce.

HTTPS

HTTPS is a protocol for **secure communication** over the internet, ensuring **data encryption** and **integrity**.

SFTP

SFTP is a network protocol for **secure file access, transfer,** and **management** over a reliable data stream using SSH.

BASIC AUTH

Basic access authentication uses a **username** and **password** provided by the HTTP user agent to **authorize requests** during an HTTP transaction.

OAUTH

OAuth is an **open standard** that allows users to grant websites or applications access to their data on other platforms **without sharing passwords**.

SIGNING REQUEST at TLS LAYER

TLS **encrypts internet data**, such as passwords or credit card details, and ensures security through a **handshake process** involving private **key-based signing requests**.

SIGNING SOAP REQUESTS

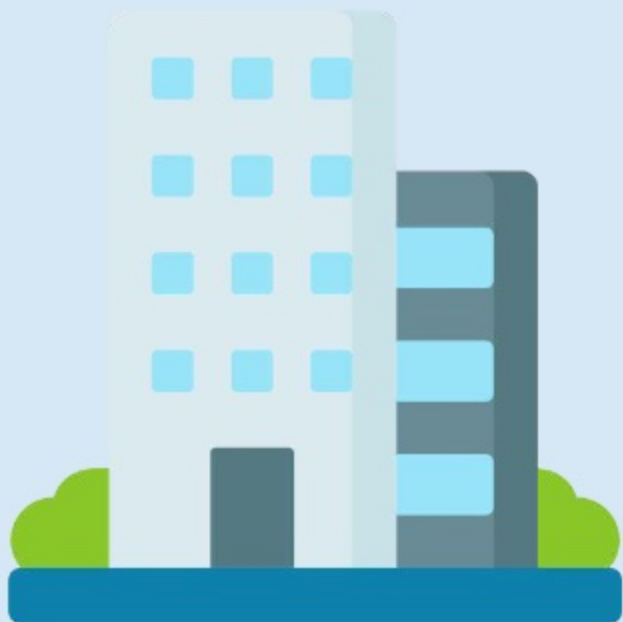
SOAP messages, structured as XML files, can include **signed elements** in their header or body to ensure **message integrity** and prevent **unauthorized modifications**.

Business Scenario

Table of Contents 

Business Scenario

The business scenario below provides details for determining the **most appropriate solutions** for the **company's requirements**.



Cosmic Express is a multinational company that provides shipping, freight and courier services. It serves both individual and corporate customers in several countries, including the United States and several European countries. It uses Salesforce to store customers' account information, contacts, cases, services offered by the company, and shipment data.

It also uses other enterprise systems, including a Transaction Processing System (TPS) for the processing of business transactions online and in company stores. A Freight Management System (FMS) is used for delivery planning, route optimization, shipment tracking, and notifications.

The IT department of the company is currently working on an integration project that aims to integrate the enterprise systems with one another. The solution architect has prepared a working draft of the solution design document, but it is missing information about the standards, components, techniques, and security mechanisms relevant to the integration solutions.

Requirements & Solutions

[Table of Contents](#)



Requirement 1



SCENARIO

A system administrator currently moves new shipment data from the Freight Management System (FMS) to Salesforce every week for reporting and analytics. The CTO would like to automate this process by creating a custom object record in Salesforce when a new shipment is created in the FMS. The FMS supports automated processes that perform HTTP requests but not SOAP API calls, so the use of REST API has been recommended for this use case. However, the solution architect still needs to define the standards and security mechanisms required for implementing the solution.



Solution 1



SOLUTION

The automated process in the FMS that needs to make REST API calls to Salesforce can be configured to use **OAuth 2.0** or **username and password** for **authentication**. Once a valid **session ID** has been obtained, it can make a REST API call using the **POST** method to create a new shipment record. The data for the record can be supplied in a **JSON request body**. The endpoint and headers of the HTTP request should be as follows:

Endpoint: https://MyDomainName.my.salesforce.com/services/data/v60.0/objects/Shipment__c/

Headers:

- a) "Authorization: Bearer sessionIdToken"
- b) "Content-Type: application/json"



Requirement 2



SCENARIO

When a new order is placed in the Transaction Processing System (TPS), certain actions should be performed in Salesforce automatically. An order related to the customer's account should be created, a log record should be generated, and an email notification should be sent to the account manager. The TPS has a custom application that supports SOAP API calls and can be triggered automatically when an order is created. The solution architect needs to define the standards, components, techniques, and security mechanisms to invoke an Apex web service in Salesforce for this use case.



Solution 2



SOLUTION

The TPS should be able to consume the WSDL file of the **custom Apex web service** in Salesforce to generate client stubs that allow invocation of the web service. The WSDL file can be generated on the '**Apex Classes**' page in Setup. The web service can perform all the necessary actions required for the integration. However, an **OAuth trust** must be established for invoking methods of the web service. A **session ID** can be obtained via a **SOAP API login() call** using valid credentials to establish an authenticated session with Salesforce. (Please note that SOAP API login() will be retired in Summer '27.)



Requirement 3



SCENARIO

If an account record of a company with a complete shipping address is created in Salesforce, account details should be sent to the FMS (Freight Management System) to allow it to determine the best route to the account's address for any future deliveries. The FMS can receive REST API calls. A record-triggered flow that executes an asynchronous HTTP callout has been recommended by the solution architect for this use case. Any related considerations must be addressed.



Solution 3



SOLUTION

The HTTP callout can use the **POST method** to send account details to the FMS. An **asynchronous path** can be included in the flow to send details once the original transaction for the triggering record has been successfully committed. An HTTP callout to a remote system uses **one-way SSL by default**. A **self-signed certificate** can be generated in Salesforce to support **two-way authentication** for the HTTP callout. Alternatively, a **certificate signed by a certificate authority (CA)** can be used. The certificate can be integrated with Apex code as follows:

```
HttpRequest request = new HttpRequest();  
request.setClientCertificateName('UniqueCertName');
```



Requirement 4



SCENARIO

The sales director of the company intends to use Salesforce to store transaction data for the purpose of analysis. A custom big object will be used to store all transaction data of the orders placed during the previous fiscal year. A subset of the data required for reporting will be moved to a custom object. The solution architect needs to outline the important considerations for populating a custom big object using Bulk API 2.0 for this use case.



Solution 4



SOLUTION

Like other Salesforce APIs, an OAuth trust must be established when using Bulk API 2.0. A third-party tool such as Postman can be used to authenticate with Salesforce. **OAuth 2.0** can be used to get an **access token** that is valid for a certain duration. Using the token, a bulk job can be created using the **POST HTTP** method. Once the job is created, data can be inserted into the custom big object using the **PUT** method. The **PATCH** method can be used to close the job, and its status can be checked via **Bulk API 2.0 query endpoints**, or on the **Bulk Data Load Jobs** page in Salesforce Setup.



Learn More

-  [Integration Patterns and Practices](#)
-  [Get to Know the Salesforce Lightning Platform APIs](#)
-  [Explore Integration Approaches](#)
-  [Ensure Integration Security](#)